

Plano do Homework – Grupo 38

Oct 9, 2026 · @Diogo

Objetivo e foco

Entregar o homework na sexta Oct 23, 2026, depois da última reunião: são 3 dias antes do prazo (Oct 26, 2026, 17:59) e valem +0,45 de bônus. O homework é uma versão compacta do projeto final, por isso tudo o que fizemos agora deve ser reutilizável.

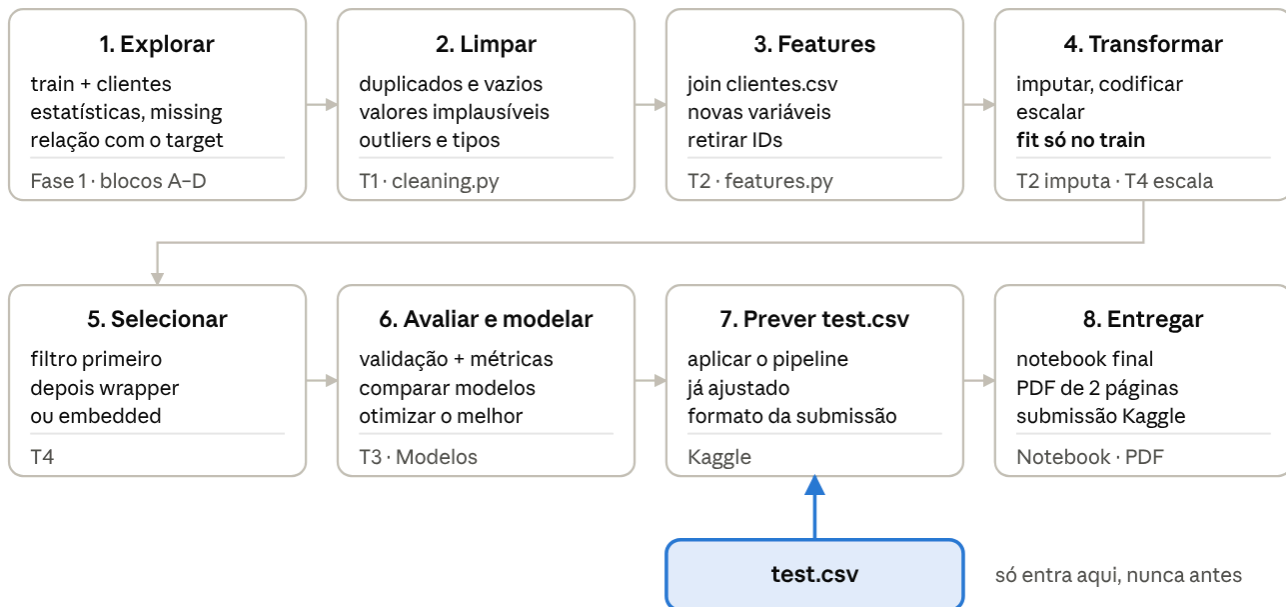
Entregável	Pontos	O que conta
Notebook – 1. Importar e explorar	3	estatísticas, qualidade dos dados, relações uni e multivariadas, insights explicados
Notebook – 2. Limpar e pré-processar	5	missing, valores implausíveis, categóricas, decisão sobre clientes.csv, novas features, scaling; nada ajustado no test
Notebook – 3. Seleção de features	3	estratégia com métodos dados nas aulas; olhar crítico sobre variáveis tipo identificador
Notebook – 4. Modelo e avaliação	4 (+1)	estratégia de avaliação, métrica justificada, previsões para todo o test.csv; +1 se ficarmos no top 5 do Kaggle
PDF de 2 páginas	5	esquema do pipeline e porque é que cada passo é adequado

Foco: 15 dos 20 pontos são explicação e rigor, não performance. Prioridade: (1) um notebook que corre do início ao fim sem erros, (2) cada decisão justificada com os nossos dados, (3) só depois afinar o modelo para o Kaggle.

Regras que não podemos falhar: só scikit-learn (sem AutoML nem Lazy Predict); o test.csv nunca é usado para ajustar transformações; nada de citar LLMs ou Medium; não incluir nada que algum de nós não saiba explicar na discussão.

Pipeline

Tudo é ajustado no train; o test.csv só entra na previsão



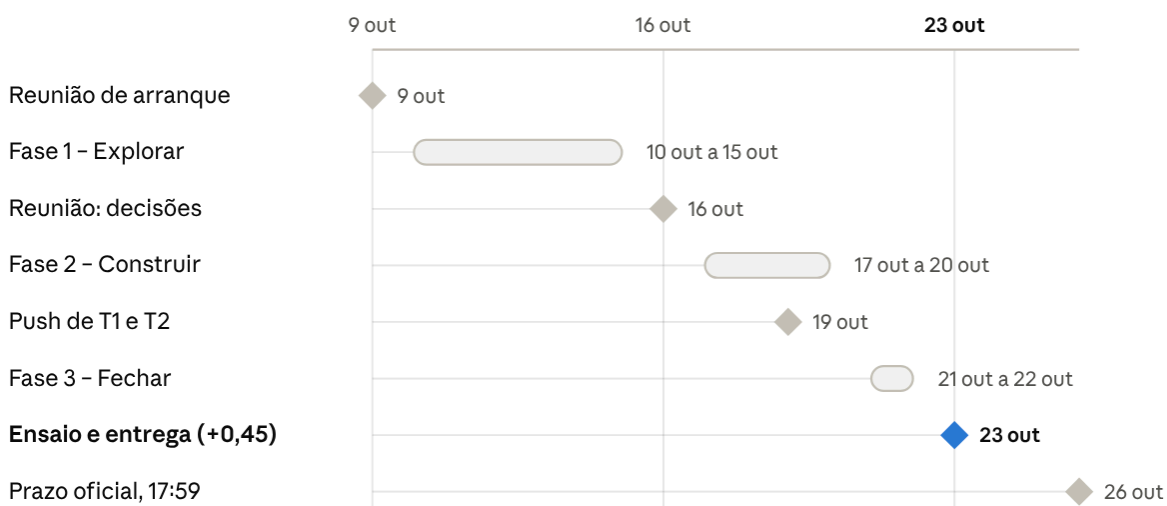
pipeline do homework · 8 etapas

As etapas 2 a 5 são ajustadas só com o train.csv e depois aplicadas tal como estão ao test.csv. Cada etapa corresponde a uma secção do notebook final e a uma caixa do esquema do PDF.

Calendário

Entregamos a 23 de outubro, três dias antes do prazo

Linhas verticais = reuniões de sexta



calendário · 3 fases, 3 reuniões

Cada fase acaba numa sexta, para a reunião servir de revisão. Os dias 24 a 26 ficam como margem: cada dia que usármos custa 0,15 de bónus.

Divisão de tarefas

As tarefas seguem as 7 secções do notebook do Grupo 38 de 2024-25 (Imports, Exploração, Pré-processamento, Feature Engineering, Seleção, Modelação, Deployment), adaptadas às regras deste ano. Cada pessoa passa por uma área diferente em cada fase e as quatro tarefas de cada fase correm em simultâneo. O sorteio foi aleatório, com uma regra: ninguém continua o mesmo tema de uma fase para a seguinte.

Estrutura do notebook final

Secção	O que inclui	Responsável
1. Imports	bibliotecas, os 4 ficheiros, semente aleatória, índice com links	Raquel (fase 3)
2. Exploração	estrutura, estatísticas, gráficos, insights por bloco de variáveis	Todos (fase 1, blocos A-D)
3.1-3.6 Limpeza	índice (ID), variáveis constantes, duplicados, diagnóstico de missing, tipos, inconsistências e valores implausíveis	Raquel (T1)
3.7 Divisão treino/validação	feita antes de qualquer fit; respeita a ordem temporal	Martim (T3)
3.8 Outliers	deteção (IQR, regras manuais), decisão de remover, limitar ou manter, só no train	Raquel (T1)
3.9 Imputação	missing com significado + flags; restantes com imputer ajustado no train	Margarida (T2)
4. Feature Engineering	join do clientes.csv, novas variáveis, retirar colunas tipo identificador	Margarida (T2)
5. Encoding, scaling e seleção	encoding e scaling; filtro (Spearman, χ^2), wrapper (RFE), embedded (árvore, Lasso) e votação final	Diogo (T4)

Secção	O que inclui	Responsável
6. Modelação e avaliação	métrica, validação cruzada e baseline (Martim, T3); comparação de modelos e GridSearch (Margarida, fase 3)	Martim + Margarida
7. Deployment	refit no train completo, previsão do test.csv, ficheiro de submissão	Diogo (fase 3)

Quem faz o quê em cada fase

Pessoa	Fase 1 – Explorar (secção 2)	Fase 2 – Construir (secções 3–5)	Fase 3 – Fechar (6, 7 e PDF)
Diogo	D – Rolling e tempo	T4 – Secção 5: encoding, scaling e seleção	Secção 7: Deployment e Kaggle
Martim	A – Transação atual	T3 – Secções 3.7 e 6: validação, métrica e baseline	PDF de 2 páginas
Margarida	C – Histórico do cliente	T2 – Secções 3.9 e 4: imputação e features	Secção 6: modelos e otimização
Raquel	B – Categóricas e clientes.csv	T1 – Secções 3.1–3.6 e 3.8: limpeza e outliers	Secção 1 e notebook final

Fase 1 – Explorar (10 a 15 out)

Cada pessoa faz a exploração completa de um bloco de variáveis, no seu próprio notebook (eda/eda_<bloco>_<nome>.ipynb).

- **A – Transação atual:** valores monetários (totais, imposto, portes, descontos, custo), detalhe da compra, distribuição do target, coluna ID.
- **B – Categóricas e clientes.csv:** pagamento, vendedor, zona, transporte, armazém, série; todo o clientes.csv e a qualidade do join por customer_id.
- **C – Histórico do cliente:** recency, tenure, número de compras anteriores, gaps históricos, gasto e valores anteriores; olhar crítico sobre customer_id.
- **D – Rolling e tempo:** contagens e gasto a 30/90/180/365 dias, random_noise, purchase_date e comparação train vs test.

O que cada bloco entrega até quinta à noite:

- ☐ Estatísticas descritivas, missing e valores implausíveis do bloco
- ☐ Gráficos univariados e relação de cada variável com o target

- ☐ Redundâncias dentro do bloco (correlações)
- ☐ Tabela “problema → ação proposta” para a limpeza
- ☐ 3 insights principais, escritos para apresentar na sexta

Fase 2 – Construir (17 a 20 out)

Na reunião de dia 16 fixamos o nome, as entradas e as saídas de cada função. Assim cada pessoa trabalha no seu ficheiro sem esperar pelos outros.

- **T1 – Limpeza e outliers, secções 3.1–3.6 e 3.8** (Raquel, `src/cleaning.py`): `clean(df)` remove duplicados e linhas vazias, transforma valores implausíveis em NaN e acerta tipos. Para os outliers, compara IQR com regras manuais e propõe o tratamento, sempre ajustado só no train.
- **T2 – Imputação e features, secções 3.9 e 4** (Margarida, `src/features.py`): `add_features(df, clientes)` faz o join, cria as novas variáveis e as flags de missing e retira as colunas tipo identificador. A imputação é ajustada no train e só aplicada à validação e ao test.
- **T3 – Validação e baseline, secções 3.7 e 6** (Martim, `src/evaluation.py`): divisão treino/validação, função que mostra as métricas de treino e de validação lado a lado (para ver overfitting) e um baseline (dummy + regressão logística). Começa com os dados em bruto e um imputer simples, por isso não depende de ninguém.
- **T4 – Encoding, scaling e seleção, secção 5** (Diogo, `src/preprocessing.py`): ColumnTransformer com encoding e scaling ajustados no train; seleção com filtro (Spearman, χ^2), wrapper (RFE) e embedded (árvore, Lasso), e uma tabela de votação para a lista final. Usa uma limpeza temporária até T1 estar no repositório.

Até segunda à noite T1 e T2 fazem push de uma primeira versão que funcione; até terça à noite as quatro peças encaixam num pipeline único.

Fase 3 – Fechar (21 a 23 out)

- **Secção 6 – Modelos e otimização** (Margarida): comparar 2 a 3 modelos do scikit-learn com a função de T3, otimizar o melhor (célula do GridSearch comentada e modelo final com os parâmetros noutra célula) e fechar com uma conclusão em markdown.
- **Secção 7 – Deployment e Kaggle** (Diogo): treinar o modelo final em todo o train, prever todo o test.csv, validar o formato contra sample_submission.csv, submeter e registar cada submissão (data, modelo, score).
- **Secção 1 e notebook final** (Raquel): `Homework_Group38.ipynb` com índice clicável, o código dos `.py` copiado para cada secção, só células com propósito e Restart & Run All sem erros.

- **PDF de 2 páginas** (Martim): esquema do pipeline com as mesmas 7 secções e a justificação de cada passo, com base no DECISOES.md.

O que aproveitamos do notebook de 2024-25

O tema era outro (classificação multiclasse de pedidos de compensação), mas a estrutura serve. **Mantemos:** as 7 secções com índice clicável; a divisão treino/validação antes de outliers, imputação e scaling; vários métodos de seleção com votação final; métricas de treino e validação lado a lado; uma conclusão em markdown no fim de cada secção.

Mudamos:

No notebook antigo	Porque não serve este ano	O que fazemos
XGBoost e SMOTE (imblearn)	Só é permitido scikit-learn, TensorFlow, Keras, PyTorch ou Transformers	modelos do scikit-learn; rebalancear só se a fase 1 mostrar que é preciso
Divisão aleatória e RepeatedKFold sem estratificação	O test.csv tem compras posteriores às do train	validação que respeite a ordem no tempo (decidir a 16 out)
<code>X_val_feature_selection = X_train_new_features.copy()</code>	A validação era uma cópia do treino	o revisor confirma de onde vem cada conjunto
Seleção de features comentada e lista final escrita à mão	Passos que não estão no notebook não contam	a seleção corre no notebook; só o GridSearch pode ficar comentado
Previsão feita com o modelo do último fold da CV	Não usa todo o train	refit no train completo antes de prever
370 células, 89 delas de uma linha (<code>.columns</code> , <code>.dtypes</code>)	Tudo no notebook limpo tem de ter propósito	células de inspeção ficam em experiments/

Compreensão partilhada

Na discussão são os professores que escolhem quem apresenta o quê, por isso cada um de nós tem de saber explicar o trabalho todo. Usamos três mecanismos.

1. Revisor rotativo. Cada trabalho tem um revisor, que muda a cada fase. Em três fases, cada pessoa revê cada uma das outras uma vez. O revisor lê o notebook ou o ficheiro, corre-o e deixa pelo menos 2 perguntas ou sugestões até ao dia antes da reunião.

Autor	Revisor na fase 1	Revisor na fase 2	Revisor na fase 3
Diogo	Raquel	Margarida	Martim
Martim	Diogo	Raquel	Margarida
Margarida	Martim	Diogo	Raquel
Raquel	Margarida	Martim	Diogo

2. Apresentações trocadas. Na reunião de sexta quem apresenta cada trabalho é o revisor, não o autor. O autor só corrige ou completa no fim. Se o revisor não o consegue explicar, ainda não está claro o suficiente.

3. Registo de decisões (DECISOES.md). Cada decisão fica numa linha: o quê, porquê (com o número ou o gráfico que a justifica), quem e o commit. É a base do PDF e da preparação para a discussão. Todos leem o ficheiro antes de cada sexta.

Antes da entrega, cada pessoa tem de conseguir responder sozinha a estas perguntas:

- ☐ Porque escolhemos esta estratégia de validação e esta métrica?
- ☐ Como tratamos cada tipo de missing e de valor implausível, e porquê?
- ☐ Como garantimos que nenhuma transformação é ajustada com o test.csv?
- ☐ Usamos o clientes.csv? Que colunas e porquê?
- ☐ Que features criámos e que features retirámos (incluindo IDs e random_noise)?
- ☐ Porque é que o modelo final ganhou aos outros, e o que faz cada hiperparâmetro afinado?

Trabalho em paralelo no Git

A regra principal: **cada ficheiro tem um único dono por fase.** Os notebooks são JSON e os conflitos neles são quase impossíveis de resolver, por isso ninguém edita o notebook de outra pessoa. Comentários e sugestões vão para uma issue no GitHub ou para o DECISOES.md.

Estrutura do repositório:

```

ml_group38/
├── data/                # nunca se altera
├── eda/                 # fase 1: um notebook por bloco
├── src/                 # fase 2: um ficheiro .py por dono
│   ├── cleaning.py
│   ├── features.py
│   ├── evaluation.py
│   └── preprocessing.py
├── experiments/         # testes livres: <nome>_<tema>.ipynb
├── submissions/         # ficheiros enviados ao Kaggle
├── Homework_Group38.ipynb # fase 3: notebook final
├── DECISOES.md
└── requirements.txt

```

Regras do dia a dia:

1. `git pull` antes de começar a trabalhar, sempre.
2. Commits pequenos e com mensagens claras (“eda C: missing de prior_mean_gap_days”), e pelo menos um push por dia de trabalho.
3. Só se faz commit de um notebook depois de Restart & Run All sem erros.
4. Mesma semente aleatória em tudo (`random_state=42`) e mesmas versões das bibliotecas (`requirements.txt`).
5. Cada linha nova no DECISOES.md fica na secção de quem a escreve, para o Git conseguir juntar as alterações sozinho.
6. Adicionar ao `.gitignore`: `.DS_Store` e `.ipynb_checkpoints/`.

Os ficheiros `.py` são só para trabalhar em paralelo na fase 2. Na fase 3 o código deles é copiado para o notebook final, porque o enunciado pede notebooks e os professores vão corrê-lo do início ao fim.

Reuniões de sexta

Cada reunião dura cerca de 1h15 e segue sempre a mesma ordem:

1. Estado do calendário e bloqueios (5 min).
2. Apresentações trocadas: cada revisor apresenta o trabalho que reviu (4 × 10 min).
3. Decisões: fechamos o que passa para o DECISOES.md (15 min).
4. Próxima fase: confirmar tarefas, revisores e as entradas e saídas de cada peça (10 min).

O que cada reunião tem de decidir:

Reunião	Tem de sair daqui
Oct 9, 2026 – arranque	Plano aprovado, tarefas da fase 1, estrutura do repositório, equipa criada no Kaggle
Oct 16, 2026 – fim da exploração	Regras de limpeza, uso do clientes.csv, estratégia de validação e métrica, nomes e assinaturas das funções da fase 2
Oct 23, 2026 – ensaio e entrega	Restart & Run All ao vivo, cada pessoa explica uma secção que não fez, revisão do PDF, submissão

Para decidir hoje

- ☐ Data de entrega: dia 23 (+0,45, com a reunião de ensaio) ou dia 21 (+0,75, mas sem ensaio em grupo)?
- ☐ Que métrica usa o leaderboard do Kaggle, e quantas submissões por dia são permitidas?
- ☐ Quem cria a equipa no Kaggle e com que nome?
- ☐ Entregamos um notebook único ou um zip de notebooks? Podemos juntar ficheiros .py? (perguntar ao professor; até haver resposta, notebook único)
- ☐ Quem cria hoje as pastas, o DECISOES.md, o requirements.txt e o .gitignore no repositório?
- ☐ Uma conversa curta a meio da semana (ex.: terça, 20 min) para a integração da fase 2, ou só por mensagem?